

# Gorgo Go for Java Programmers — Answer Key

May 28, 2026

## About This Answer Key

Work out the answers yourself before looking here. The explanations are meant to help you understand *why* the answer is what it is, not just tell you *what* the answer is.

---

## Chapter 0: How to Use This Booklet

There are no exercises in Chapter 0.

---

## Chapter 1: Hello, Go

**Exercise 1** (Think about it): Go has no `protected` access modifier. Java uses `protected` to allow subclasses in other packages to access members. Go has no inheritance. What Go mechanism serves a similar purpose when you want controlled access across packages, and what are its limits compared to `protected`?

Go’s closest substitute is the `internal` directory convention. Any package placed under an `internal/` directory can only be imported by code rooted at the parent of that `internal/` tree. For example, `github.com/myorg/myapp/internal/auth` is visible to `github.com/myorg/myapp` and its subpackages, but not to any external module.

The limits are real: `internal` is a coarse, directory-level gate, not a per-symbol toggle. You cannot mark a single exported function as “visible only to sibling packages” the way Java’s `protected` marks a single method as “visible to subclasses in other packages.” If you need a symbol visible to several packages in your module but hidden from the outside world, you put it in `internal/`; everything in that subtree shares the same level of access. There is also no concept of subclass access because Go has no subclasses — embedding a struct gives you promoted methods, but embedding is not inheritance and carries no visibility privileges.

---

**Exercise 2** (What does this print?):

```
package main

import "fmt"

func main() {
    name := "Chappell Roan"
    plays := 1_500_000
    fmt.Printf("%s has %d plays\n", name, plays)
    fmt.Printf("type of plays: %T\n", plays)
```

```
    fmt.Println(fmt.Sprintf("quoted: %q", name))
}
```

Output:

```
Chappell Roan has 1500000 plays
type of plays: int
quoted: "Chappell Roan"
```

`%s` formats the string without quotes. `%d` formats the integer in base 10; the `_` digit separator in the source literal `1_500_000` is purely cosmetic — the value is 1500000. `%T` prints the Go type name, which for an integer literal assigned with `:=` is `int`. `%q` wraps the string in double quotes and escapes any characters that need it. `fmt.Sprintf` returns the formatted string; `fmt.Println` then prints it with a newline appended.

---

**Exercise 3** (Calculation): A Go source file imports three packages: `"fmt"`, `"os"`, and `"math"`. The code calls `fmt.Println` and `os.Exit`, but never calls anything from `math`. How many compiler errors does this program produce, and which import causes them?

One compiler error. Go emits one error per unused import, not one per symbol. The error points to the `"math"` import:

```
./main.go:5:2: "math" imported and not used
```

`"fmt"` and `"os"` are used, so they cause no error. The build fails entirely — there is no “compile with warnings” mode in Go.

---

**Exercise 4** (Where is the bug?):

```
package main

import (
    "fmt"
    "math"
)

func main() {
    fmt.Println("Hello, Go!")
}
```

`"math"` is imported but never used. The program will not compile. The compiler produces:

```
./main.go:5:2: "math" imported and not used
```

The fix is to remove the `"math"` import. If you were writing this in an editor with `goimports` configured to run on save, it would have removed the unused import automatically before you even tried to build.

---

**Exercise 5** (Write a program): Write a Go program that declares a `const` string with a song title of your choice and prints three lines: the title using `%s`, the title using `%q`, and the Go type of the title using `%T`. Run it with `go run`.

```
package main

import "fmt"

const title = "Good Luck, Babe!"
```

```
func main() {
    fmt.Printf("%s\n", title)
    fmt.Printf("%q\n", title)
    fmt.Printf("%T\n", title)
}
```

Output:

```
Good Luck, Babe!
"Good Luck, Babe!"
string
```

`const` values are untyped string constants when declared without an explicit type. `%T` prints `string` because the constant's default type is `string`. Note that `%q` adds the double quotes and would also escape any special characters inside the string, such as newlines or backslashes.

---

## Chapter 2: Types and Variables

**Exercise 1** (Think about it): In Java, using an uninitialized local variable is a compile error. In Go, every variable has a zero value. What are the benefits and potential risks of zero values?

The primary benefit is safety and predictability: there are no uninitialized reads and no undefined behavior from reading a garbage value off the stack. You can declare a `sync.Mutex` or a `bytes.Buffer` and use it immediately without calling a constructor, because the zero value is deliberately designed to be a valid, ready-to-use state. This is a Go idiom worth internalizing: if you design a type so that its zero value is useful, callers pay no initialization tax.

The risk is silent logic errors. In Java, forgetting to initialize a variable is a compile-time catch. In Go, `var count int` silently starts at 0, `var name string` at "", and `var ptr *SomeType` at `nil`. If you forget to set `ptr` before dereferencing it, you get a runtime panic, not a compile error. Similarly, a `bool` zero value is `false`, so a field like `IsAdmin` starts as `false` — which is the safe default — but a field like `IsEnabled` also starts as `false`, which might not be what you want. Zero values encourage a different discipline: design your data so that the zero state is meaningful and correct.

---

**Exercise 2** (What does this print?):

```
package main

import "fmt"

type StreamingTier int

const (
    Free StreamingTier = iota
    Standard
    Premium
    Lossless
)

func main() {
    fmt.Println(Free, Standard, Premium, Lossless)
    fmt.Printf("Premium = %d\n", Premium)
}
```

Output:

```
0 1 2 3
Premium = 2
```

`iota` starts at 0 for the first constant in a `const` block and increments by 1 for each subsequent constant. `Free` gets 0, `Standard` gets 1, `Premium` gets 2, and `Lossless` gets 3. Because `StreamingTier` is a named type based on `int`, `%d` formats it as a plain integer. `fmt.Println` with multiple arguments separates them with spaces and appends a newline.

---

**Exercise 3** (Calculation): What is the zero value of each of the following types: `int`, `float64`, `bool`, `string`, `*int`?

| Type                 | Zero value                  |
|----------------------|-----------------------------|
| <code>int</code>     | 0                           |
| <code>float64</code> | 0.0                         |
| <code>bool</code>    | false                       |
| <code>string</code>  | "" (empty string, length 0) |
| <code>*int</code>    | nil                         |

Every variable declared with `var` and no initializer gets its type's zero value. `nil` is the zero value for all pointer types, as well as slices, maps, channels, functions, and interfaces. A `nil` pointer is safe to declare but will panic if you dereference it.

---

**Exercise 4** (Where is the bug?):

```
package main

import "fmt"

func main() {
    x := 10
    x := 20
    fmt.Println(x)
}
```

`:=` requires at least one new variable on the left side. Because `x` is already declared in the same scope, the second `:=` is a compile error:

```
./main.go:7:4: no new variables on left side of :=
```

The fix depends on intent. To reassign `x`, use a plain `=`:

```
x := 10
x = 20
fmt.Println(x)
```

`:=` is valid the second time only if you are introducing at least one new variable alongside the existing one, for example `x, y := 20, 30`.

---

**Exercise 5** (Write a program): Define types `Celsius` and `Fahrenheit` based on `float64`. Write conversion functions. Print the boiling and freezing points of water in both scales.

```

package main

import "fmt"

type Celsius float64
type Fahrenheit float64

func CToF(c Celsius) Fahrenheit {
    return Fahrenheit(c*9/5 + 32)
}

func FToC(f Fahrenheit) Celsius {
    return Celsius((f - 32) * 5 / 9)
}

func main() {
    freezingC := Celsius(0)
    boilingC := Celsius(100)

    fmt.Printf("Freezing: %.1f°C = %.1f°F\n", freezingC, CToF(freezingC))
    fmt.Printf("Boiling:   %.1f°C = %.1f°F\n", boilingC, CToF(boilingC))

    freezingF := Fahrenheit(32)
    boilingF := Fahrenheit(212)

    fmt.Printf("Freezing: %.1f°F = %.1f°C\n", freezingF, FToC(freezingF))
    fmt.Printf("Boiling:   %.1f°F = %.1f°C\n", boilingF, FToC(boilingF))
}

```

Output:

```

Freezing: 0.0°C = 32.0°F
Boiling:  100.0°C = 212.0°F
Freezing: 32.0°F = 0.0°C
Boiling:  212.0°F = 100.0°C

```

The key insight is that `Celsius` and `Fahrenheit` are distinct types even though both are backed by `float64`. You cannot pass a `Celsius` where a `Fahrenheit` is expected without an explicit conversion. This is why type definitions exist: they let the compiler catch unit errors at compile time rather than at runtime.

---

## Chapter 3: Strings, Bytes, and Runes

**Exercise 1** (Think about it): If Go strings are byte sequences and not character sequences, what happens when you index into a string containing a multibyte character like `é`? How does `for range` behave differently from `for i := 0; i < len(s); i++`?

Indexing with `s[i]` yields the byte at position `i`, not the character. For ASCII, one byte is one character, so `s[0]` on `"café"` gives `'c'` as a byte (`uint8` value 99). But `é` in UTF-8 is encoded as two bytes (`0xC3 0xA9`). If `"café"` starts at index 0, then `s[3]` is `0xC3` — the first byte of `é` — not the character `é` itself. This is often surprising and is a common source of bugs when programmers index into strings that may contain non-ASCII characters.

`for i := 0; i < len(s); i++` walks byte by byte, so iterating `"café"` visits 5 bytes (`c`, `a`, `f`, and the two bytes of `é`).

for `i, r := range s` decodes the string as UTF-8 on each iteration. `i` is the byte offset of the start of the rune, and `r` is the decoded **rune** (a full Unicode code point as `int32`). Iterating `"café"` with `range` visits 4 runes: `'c'`, `'a'`, `'f'`, and `'é'`. When `range` reaches the two-byte sequence for `é`, it decodes both bytes together and advances `i` by 2. Use `range` when you care about characters; use byte indexing only when you are sure the string is pure ASCII or when you genuinely need to operate on raw bytes.

---

**Exercise 2** (What does this print?):

```
package main

import "fmt"

func main() {
    s := "café"
    fmt.Println("len:", len(s))
    fmt.Printf("s[3] = %d (0x%X)\n", s[3], s[3])

    for i, r := range s {
        fmt.Printf("index %d: %c (%d)\n", i, r, r)
    }
}
```

Output:

```
len: 5
s[3] = 195 (0xC3)
index 0: c (99)
index 1: a (97)
index 2: f (102)
index 3: é (233)
```

`"café"` is 5 bytes long: `c` `a` `f` are one byte each, and `é` is two bytes (`0xC3 0xA9`). `s[3]` returns the byte at position 3, which is `0xC3` (decimal 195) — the first byte of the UTF-8 encoding of `é`, not the character itself. `range` decodes the UTF-8 properly: it sees the two bytes at position 3 as a single rune (`é`, code point U+00E9, decimal 233) and skips byte 4 entirely. Notice that index 4 never appears in the range output because `é` occupies two byte positions (3 and 4) but is a single rune.

---

**Exercise 3** (Calculation): How many bytes does `len("Beyoncé")` return? How many runes does `utf8.RuneCountInString("Beyoncé")` return?

`len("Beyoncé")` returns 8. `B`, `e`, `y`, `o`, `n`, `c` are each one byte (6 bytes total), and `é` (U+00E9) encodes to two bytes in UTF-8, giving 8 bytes.

`utf8.RuneCountInString("Beyoncé")` returns 7. The string contains 7 characters: `B`, `e`, `y`, `o`, `n`, `c`, and `é`. `len` counts bytes; `utf8.RuneCountInString` counts Unicode code points. They agree on pure ASCII strings and diverge the moment any character requires more than one byte.

---

**Exercise 4** (Where is the bug?):

```
package main

import "fmt"

func shout(s string) string {
```

```

    result := make([]byte, len(s))
    for i := 0; i < len(s); i++ {
        result[i] = s[i] - 32
    }
    return string(result)
}

func main() {
    fmt.Println(shout("café"))
}

```

The function assumes every character is a single byte and that subtracting 32 uppercases it. Both assumptions are wrong for non-ASCII input.

For ASCII letters, subtracting 32 from a lowercase byte gives the corresponding uppercase byte (e.g., 'a' - 32 = 'A'). But "café" is 5 bytes, and bytes 3 and 4 are the UTF-8 encoding of é (0xC3 and 0xA9). Subtracting 32 from each gives 0xA3 and 0x89, which are not a valid UTF-8 sequence for any meaningful character. The result is garbled output or a replacement character, not "CAFÉ".

The correct approach for Unicode-aware uppercasing is `strings.ToUpper`:

```

import "strings"

func shout(s string) string {
    return strings.ToUpper(s)
}

```

Output:

CAFÉ

---

**Exercise 5** (Write a program): Write a program that reverses a string correctly — by rune, not by byte — and prints the result.

```

package main

import "fmt"

func reverseString(s string) string {
    runes := []rune(s)
    for i, j := 0, len(runes)-1; i < j; i, j = i+1, j-1 {
        runes[i], runes[j] = runes[j], runes[i]
    }
    return string(runes)
}

func main() {
    words := []string{"café", "Beyoncé", "hello"}
    for _, w := range words {
        fmt.Printf("%q -> %q\n", w, reverseString(w))
    }
}

```

Output:

```

"café" -> "éfac"
"Beyoncé" -> "écnoyeB"

```

```
"hello" -> "olleh"
```

The key step is converting the string to `[]rune` first. This decodes the UTF-8 and gives you one `rune` per Unicode code point regardless of how many bytes each one occupies. You then swap elements in the rune slice in place, and convert back to a `string` at the end. Reversing the raw `[]byte` instead would shuffle the individual bytes of multibyte characters and produce invalid UTF-8.

---

## Chapter 4: Control Flow

**Exercise 1** (Think about it): Go has only one loop keyword, `for`. Java has `for`, `while`, and `do...while`. Is this limiting, or does it simplify the language? Can you think of a pattern where a `do...while` loop cannot be elegantly expressed with Go's `for`?

Having one loop keyword is not limiting in practice. Go's `for` covers all three Java forms:

- C-style `for`: `for i := 0; i < n; i++ { ... }`
- while-style: `for condition { ... }`
- infinite: `for { ... }` with a `break` inside

The `do...while` pattern — execute the body at least once, then check the condition — requires a small idiom in Go:

```
for {
    // body
    if !condition {
        break
    }
}
```

This is slightly less elegant than Java's `do { ... } while (condition)` because the loop-exit logic is inside the body rather than at the bottom of the statement. For readers scanning code quickly, the exit condition is less visible. In practice, `do...while` is rare enough in both languages that this is a minor inconvenience. The benefit is a smaller language: one keyword to teach, one loop construct to remember, and fewer edge cases around scoping and control flow.

---

**Exercise 2** (What does this print?):

```
package main

import "fmt"

func main() {
    for i := 0; i < 3; i++ {
        defer fmt.Println(i)
    }
    fmt.Println("done")
}
```

Output:

```
done
2
1
0
```



Two things are happening here. First, `defer` runs after the surrounding function returns, so all three deferred calls happen after `fmt.Println("done")`. Second, `defer` arguments are evaluated immediately at the point of the `defer` statement, not when the deferred call executes. When `i` is 0, `defer fmt.Println(0)` is registered with the value 0 baked in. When `i` is 1, `defer fmt.Println(1)` is registered with 1. When `i` is 2, `defer fmt.Println(2)` is registered with 2. Deferred calls execute in LIFO (last-in, first-out) order, so the last one registered runs first: 2, then 1, then 0.

---

**Exercise 3** (What does this print?):

```
package main

import "fmt"

func grade(score int) string {
    switch {
    case score >= 90:
        return "A"
    case score >= 80:
        return "B"
    case score >= 70:
        return "C"
    default:
        return "F"
    }
}

func main() {
    fmt.Println(grade(95))
    fmt.Println(grade(83))
    fmt.Println(grade(70))
    fmt.Println(grade(55))
}
```

Output:

A  
B  
C  
F

An expression-less `switch` is equivalent to `switch true` — each case is a boolean expression, and the first one that evaluates to `true` wins. Cases are evaluated top to bottom; once a match is found, the remaining cases are skipped. There is no fallthrough, so `grade(83)` matches `score >= 80`, returns "B", and never reaches `score >= 70`. The `default` clause matches when no case is true.

---

**Exercise 4** (Where is the bug?):

```
package main

import "fmt"

func makeMultipliers() []func(int) int {
    fns := make([]func(int) int, 3)
    factor := 1
```

```

    for i := 0; i < 3; i++ {
        factor = (i + 1) * 10
        fns[i] = func(x int) int { return x * factor }
    }
    return fns
}

func main() {
    fns := makeMultipliers()
    for _, f := range fns {
        fmt.Println(f(5))
    }
}

```

All three calls print 150, not 50, 100, 150.

The bug is that `factor` is declared outside the loop. All three closures capture the same `factor` variable by reference. By the time the closures run, the loop has finished and `factor` is 30 (the last value assigned). Every closure multiplies by 30, so `f(5)` returns 150 for all three.

The fix is to declare `factor` inside the loop so each iteration gets its own copy:

```

for i := 0; i < 3; i++ {
    factor := (i + 1) * 10
    fns[i] = func(x int) int { return x * factor }
}

```

Now each closure captures a distinct `factor` variable, and the output is 50, 100, 150.

---

**Exercise 5** (Write a program): Write a program using `defer` and `recover` to catch a panic from a function that calls `panic("something went wrong")`, print the recovered message, and continue execution normally.

```

package main

import "fmt"

func safeRun(f func()) {
    defer func() {
        if r := recover(); r != nil {
            fmt.Println("recovered:", r)
        }
    }()
    f()
}

func riskyOperation() {
    fmt.Println("starting risky operation")
    panic("something went wrong")
}

func main() {
    fmt.Println("before safeRun")
    safeRun(riskyOperation)
    fmt.Println("after safeRun --- execution continues normally")
}

```

Output:

```
before safeRun
starting risky operation
recovered: something went wrong
after safeRun --- execution continues normally
```

`recover` only works when called directly inside a deferred function. The deferred anonymous function runs when `riskyOperation` panics; at that point, `recover()` returns the value passed to `panic` and stops the panic from propagating up the call stack. After `safeRun` returns normally, `main` continues.

Note that `panic` and `recover` are not Go's primary error handling mechanism — that role belongs to returning `error` values. Use `panic/recover` only for truly unrecoverable situations or at the boundary of a library to prevent a panic from escaping into caller code.

---

## Chapter 5: Functions — Answers

**Exercise 1** (Think about it): Go returns errors as values rather than throwing exceptions. A Java checked exception forces the caller to handle it — the compiler will not let you ignore it. Go's multi-return error is also explicit, but you can discard it with `_` or simply not assign the second return value. Does Go's approach give you the same safety guarantee as Java's checked exceptions? What is gained and what is lost by each approach?

Java's checked exceptions provide a compiler-enforced contract: if a method declares `throws IOException`, the caller must either catch it or declare that it also throws it. There is no way to silently ignore a checked exception in Java without at least writing a catch block (even an empty one is conspicuous).

Go does not provide that same guarantee. You can write `result, _ := divide(a, b)` or even `result := somePackage.Lookup(key)` if you know (or ignore) that `Lookup` returns `(string, error)` but only assign one variable — though the compiler will reject an assignment that captures the wrong number of values, it will happily accept `_` for any of them.

**What Go gains:**

- Error handling is explicit code, not a separate control-flow mechanism. Errors flow through the same call stack as regular values, which makes them easier to wrap, annotate, and inspect.
- There is no distinction between checked and unchecked exceptions to manage. In Java, many APIs force you to wrap checked exceptions in `RuntimeException` just to use them in lambdas or streams.
- Errors are plain values you can store, compare, pass around, and test — no reflection required.

**What Java gains:**

- The compiler can prove at build time that every failure path is addressed. Go relies on code review and tools like `errcheck` to catch ignored errors.
- Stack traces are attached to exceptions automatically; in Go you must explicitly wrap errors with `fmt.Errorf("...: %w", err)` to build a chain.

In practice, Go's approach leads to more explicit error-handling code and fewer “surprise” failures from unchecked exceptions — but it also produces more repetitive `if err != nil` checks that disciplined engineers must not skip.

---

**Exercise 2** (What does this print?):

```
package main

import "fmt"
```

```
func makeAdder(n int) func(int) int {
    return func(x int) int {
        return x + n
    }
}
```

```
func main() {
    add5 := makeAdder(5)
    add10 := makeAdder(10)
    fmt.Println(add5(3))
    fmt.Println(add10(3))
    fmt.Println(add5(add10(1)))
}
```

Output:

```
8
13
16
```

`makeAdder(5)` returns a closure that captures `n = 5`. `makeAdder(10)` returns a separate closure that captures `n = 10`. Each closure has its own independent copy of `n` because each call to `makeAdder` creates a new variable.

`add5(3)` returns  $3 + 5 = 8$ . `add10(3)` returns  $3 + 10 = 13$ . `add5(add10(1))` evaluates inside-out: `add10(1)` returns  $1 + 10 = 11$ , then `add5(11)` returns  $11 + 5 = 16$ .

**Exercise 3** (Calculation): Given the function below, what values are printed by the three `fmt.Println` calls? Trace the value of `total` at each step.

```
package main

import "fmt"

func running(start int) func(int) int {
    total := start
    return func(n int) int {
        total += n
        return total
    }
}

func main() {
    acc := running(100)
    fmt.Println(acc(10))
    fmt.Println(acc(20))
    fmt.Println(acc(-5))
}
```

Output:

```
110
130
125
```

`running(100)` creates a closure that captures `total`, initialised to 100. The same `total` variable is shared across all calls through `acc` because the closure captures it by reference — it is the same memory location every time.

- `acc(10)`: `total = 100 + 10 = 110`; returns and prints 110.
- `acc(20)`: `total = 110 + 20 = 130`; returns and prints 130.
- `acc(-5)`: `total = 130 + (-5) = 125`; returns and prints 125.

This is a running total (accumulator) implemented with a closure. Each call modifies and returns the accumulated value.

---

**Exercise 4** (Where is the bug?): The following code tries to build a slice of greeting functions, one for each name in a list, using a Go 1.21 module.

```
package main

import "fmt"

func main() {
    names := []string{"benSON", "amara", "priya"}
    greets := make([]func(), len(names))
    for i, name := range names {
        greets[i] = func() { fmt.Println("hola,", name) }
    }
    for _, g := range greets {
        g()
    }
}
```

**The bug:** Under Go 1.21 semantics, the `for range` loop reuses the same `i` and `name` variables for every iteration. All three closures capture the same `name` variable — not a copy of its value, but a reference to the single loop variable. By the time any of the closures run, the loop has finished and `name` holds the last value: `"priya"`.

Actual output under Go 1.21:

```
hola, priya
hola, priya
hola, priya
```

Expected output:

```
hola, benSON
hola, amara
hola, priya
```

**Fix 1 (works in all Go versions):** Create a new local variable inside the loop body to shadow the loop variable:

```
for i, name := range names {
    name := name // create a new variable for this iteration
    greets[i] = func() { fmt.Println("hola,", name) }
}
```

**Fix 2 (works in Go 1.22+):** Update the `go` directive in `go.mod` to `go 1.22` or later. The language change makes each loop iteration create its own variable automatically, so all existing closure-in-loop code behaves correctly without any source changes.

**Exercise 5** (Write a program): Write a function `pipeline(fns ...func(int) int) func(int) int` that takes any number of `func(int) int` functions and returns a new function that applies them in order.

```
package main

import "fmt"

// pipeline returns a function that applies each fn in fns left to right.
func pipeline(fns ...func(int) int) func(int) int {
    return func(x int) int {
        result := x
        for _, fn := range fns {
            result = fn(result) // apply each transform in sequence
        }
        return result
    }
}

func main() {
    double := func(n int) int { return n * 2 } // multiply by 2
    addTen  := func(n int) int { return n + 10 } // add 10
    square  := func(n int) int { return n * n } // square the value

    // double then addTen: (3*2)+10 = 16
    p1 := pipeline(double, addTen)
    fmt.Println(p1(3)) // 16

    // addTen then double: (3+10)*2 = 26
    p2 := pipeline(addTen, double)
    fmt.Println(p2(3)) // 26

    // double then square: (3*2)^2 = 36
    p3 := pipeline(double, square)
    fmt.Println(p3(3)) // 36

    // empty pipeline --- identity
    p4 := pipeline()
    fmt.Println(p4(7)) // 7
}
```

Key points in this solution:

- `pipeline` is variadic: it accepts any number of `func(int) int` values.
- Inside the returned closure, `fns` is captured by reference — the same `[]func(int) int` slice the outer call built.
- The order of application matters: `pipeline(double, addTen)` and `pipeline(addTen, double)` produce different results for the same input.
- An empty `pipeline()` call returns an identity function because the loop body never executes.

---

## Chapter 6: Pointers — Answers

**Exercise 1** (Think about it): In Java, you can mutate the fields of an object passed to a method, but you cannot change which object the caller's variable refers to. In Go, you can do neither with a plain struct value

parameter, but you can do both with a pointer parameter. What is the deeper rule that unifies these two behaviors, and what does it tell you about how Go and Java model “passing by reference”?

The unifying rule is: **a function can only affect the caller through a pointer (or reference) to the thing the caller cares about.**

In Java, every object variable already is a hidden pointer to heap memory. When you pass an object to a method, you pass a copy of the reference — the method shares the same underlying object, so field mutations are visible to the caller. But the reference itself is passed by value, so reassigning the parameter inside the method (`c = new Counter()`) does not change the caller’s variable. Java gives you reference semantics for objects without exposing the pointer.

In Go, everything — including structs — is passed by value by default. A struct parameter is a copy of the whole struct; mutating its fields inside the function leaves the caller’s copy unchanged. If you want the Java-like “shared object” behavior, you pass a `*MyStruct`. Now you have an explicit pointer to the same memory, and you can mutate fields or even replace the entire struct value with `*p = MyStruct{...}`.

The models are consistent once you see the underlying mechanism: Java hides the pointer for objects; Go exposes it uniformly for everything. Neither language is “passing by reference” in the C++ sense of a reference binding to the caller’s variable — both copy something at the call site. The difference is what that something is.

---

**Exercise 2** (What does this print?):

```
package main

import "fmt"

func double(n *int) {
    *n *= 2
}

func main() {
    a := 5
    b := &a
    double(b)
    fmt.Println(a)
    fmt.Println(*b)
}
```

Output:

```
10
10
```

`b` holds the address of `a`, so `b` and `&a` point to the same integer. `double(b)` passes that pointer to `double`, which multiplies the value at that address by 2:  $5 * 2 = 10$ . After `double` returns, `a` is 10 because it is the same memory that was modified. `*b` dereferences `b`, which still points to `a`, so it also yields 10. There is only one integer in memory; `a` and `*b` are two names for the same location.

---

**Exercise 3** (Calculation): Given the following declarations:

```
x := 100
p := &x
q := p
*q = 200
```

What are the values of `x`, `*p`, and `*q` after these four lines? How many distinct integers are stored in memory? After these four lines:

| Expression      | Value |
|-----------------|-------|
| <code>x</code>  | 200   |
| <code>*p</code> | 200   |
| <code>*q</code> | 200   |

Only **one** distinct integer is stored in memory.

`p := &x` makes `p` point to `x`. `q := p` copies the pointer value itself (the address), so `q` now points to the same `x`. `*q = 200` writes 200 to the memory location that `q` points to, which is `x`. All three expressions (`x`, `*p`, `*q`) refer to the same memory location, so they all read 200. Assigning one pointer variable to another (`q := p`) never copies the pointed-to integer — it copies only the address.

---

**Exercise 4** (Where is the bug?): The following code tries to append a suffix to a string through a pointer.

```
package main

import "fmt"

func addExcitement(s *string) {
    s += "!"
}

func main() {
    msg := "Bad Guy"
    addExcitement(&msg)
    fmt.Println(msg)
}
```

The bug is on the line `s += "!"`. `s` is a `*string` — a pointer to a string. You cannot use `+=` to concatenate a string onto a pointer; that operation does not even compile. The compiler reports: `invalid operation: s += "!" (mismatched types *string and untyped string)`.

The intent was to modify the string that `s` points to. The correct approach is to dereference `s` first, then concatenate, then write the result back:

```
func addExcitement(s *string) {
    *s += "!" // dereference to get the string, concatenate, write back
}
```

With this fix the program prints:

Bad Guy!

The general rule: `s` is the pointer (the address); `*s` is the value at that address. Any operation that should work on the string must operate on `*s`, not on `s` itself.

---

**Exercise 5** (Write a program): Write a function `increment(n *int)` that adds one to the integer pointed to by `n`. Then write a second function `incrementBy(n *int, delta int)` that adds `delta` to the integer. In `main`, declare an integer variable, call both functions on it, and print the value after each call to confirm the changes took effect.



```

package main

import "fmt"

func increment(n *int) {
    *n++    // dereference n and add one to the value it points to
}

func incrementBy(n *int, delta int) {
    *n += delta    // dereference n and add delta to the value it points to
}

func main() {
    score := 0
    fmt.Println(score)    // 0

    increment(&score)
    fmt.Println(score)    // 1

    incrementBy(&score, 9)
    fmt.Println(score)    // 10

    incrementBy(&score, -3)
    fmt.Println(score)    // 7
}

```

Output:

```

0
1
10
7

```

`increment` takes `*n++`, which is parsed as `(*n)++` — it increments the integer at the address `n` holds. `incrementBy` uses `*n += delta` to add an arbitrary amount. Because both functions receive a pointer to `score`, every write through `*n` updates the original `score` variable in `main`.

---

## Chapter 7: Slices — Answers

**Exercise 1** (Think about it): In Java, an `ArrayList<Integer>` holds references to `Integer` objects on the heap. A Go `[]int` holds the integers directly in the backing array. What are the performance implications of this difference? When would you feel the difference most?

A Java `ArrayList<Integer>` stores a pointer (reference) to each boxed `Integer` object, and each `Integer` object lives somewhere on the heap. Iterating over the list means following a pointer for every element, and those `Integer` objects may be scattered around memory — poor cache locality. Each `Integer` also carries object header overhead (typically 16 bytes) even though the actual integer value is just 4 bytes.

A Go `[]int` stores the integer values **directly** and **contiguously** in the backing array. Iterating is a sequential scan through a flat region of memory: the CPU prefetcher handles this extremely well. There is no per-element allocation overhead and no pointer chasing.

You feel the difference most in:

- **Tight loops** that process large slices: the sequential memory access pattern is cache-friendly, and modern CPUs can vectorize flat integer arrays.
- **Memory usage:** a Go `[]int` of a million elements is roughly 8 MB (64-bit ints). A Java `ArrayList<Integer>` of a million elements is the list's pointer array (8 MB of references) plus a million `Integer` objects on the heap (at least 16 MB more), for a minimum of 24 MB — and GC pressure from all those small objects.
- **GC pauses:** the Go garbage collector has no pointers to trace inside a `[]int`, so it scans the array in constant time. A Java `ArrayList<Integer>` forces the GC to follow a million references.

The trade-off is that Go's approach works for slices of value types (`int`, `float64`, structs). For slices of interfaces or pointers, Go has the same indirection that Java does.

---

**Exercise 2** (What does this print?):

```
package main

import "fmt"

func main() {
    a := []int{10, 20, 30, 40, 50}
    b := a[1:4]
    b[0] = 99
    b = append(b, 77)
    fmt.Println(a)
    fmt.Println(b)
}
```

Output:

```
[10 99 30 40 77]
[99 30 40 77]
```

Step by step:

1. `a` is created with backing array `[10, 20, 30, 40, 50]`, `len=5`, `cap=5`.
2. `b := a[1:4]` creates a slice header pointing at `a[1]`, `len=3`, `cap=4` (from index 1 to the end of `a`'s backing array). `b` sees `[20, 30, 40]`.
3. `b[0] = 99` writes 99 to the backing array at `a`'s index 1. Now the backing array is `[10, 99, 30, 40, 50]`. `a` sees `[10, 99, 30, 40, 50]` and `b` sees `[99, 30, 40]`.
4. `b = append(b, 77)`: `b` has `len=3`, `cap=4`, so there is room for one more element without reallocation. `append` writes 77 to the backing array at `a`'s index 4 (the element after `b`'s current last element), and returns a new slice header with `len=4`. The backing array is now `[10, 99, 30, 40, 77]`. `a` still has `len=5` and sees `[10, 99, 30, 40, 77]`. `b` now has `len=4` and sees `[99, 30, 40, 77]`.

Both the `b[0] = 99` write and the `append(b, 77)` write go through to the shared backing array and are visible through `a`. This is the classic slice aliasing trap.

---

**Exercise 3** (Calculation):

```
s := make([]int, 3, 8)
s = append(s, 1, 2)
```

After the first line: `len(s) = 3`, `cap(s) = 8`. The backing array has eight slots; the first three are zero.

After the second line: `append` needs to add two elements to a slice with `len=3` and `cap=8`. Since  $3 + 2 = 5$ , which is less than 8, there is enough room in the existing backing array. No new backing array is allocated. `append` writes 1 and 2 at indices 3 and 4, and returns a new header with `len=5`.

Final values: `len(s) = 5`, `cap(s) = 8`. No new backing array is allocated because the capacity was sufficient.

---

**Exercise 4** (Where is the bug?):

```
package main

import "fmt"

func removeFirst(s []int) []int {
    return s[1:]
}

func main() {
    data := []int{1, 2, 3, 4, 5}
    trimmed := removeFirst(data)
    trimmed[0] = 99
    fmt.Println(data)
    fmt.Println(trimmed)
}
```

Output:

```
[1 99 3 4 5]
[99 3 4 5]
```

The bug is that `s[1:]` returns a sub-slice that **shares the backing array** with `data`. `trimmed` points at `data[1]`, so `trimmed[0]` is the same memory location as `data[1]`. Assigning `trimmed[0] = 99` visibly changes `data[1]`.

The programmer likely intended `trimmed` to be an independent copy of the data with the first element removed. The fix is to use `copy`:

```
func removeFirst(s []int) []int {
    if len(s) == 0 {
        return nil
    }
    result := make([]int, len(s)-1)
    copy(result, s[1:])
    return result
}
```

With this fix, `trimmed[0] = 99` does not affect `data` at all, and `fmt.Println(data)` prints `[1 2 3 4 5]`.

Alternatively, if you want the three-index form to at least prevent accidental `append` overwrites (while still sharing memory for reads), you could write `s[1:len(s):len(s)]`. But if true independence is required, `copy` is the right tool.

---

**Exercise 5** (Write a program):

```
package main

import "fmt"
```

```

func unique(s []string) []string {
    seen := make(map[string]bool)
    result := make([]string, 0, len(s))
    for _, v := range s {
        if !seen[v] {
            seen[v] = true
            result = append(result, v)
        }
    }
    return result
}

func main() {
    input := []string{"pop", "indie", "pop", "R&B", "indie"}
    out := unique(input)
    fmt.Println(out) // [pop indie R&B]

    // Confirm independence: modifying out does not touch input.
    out[0] = "classical"
    fmt.Println(input[0]) // pop --- unchanged
    fmt.Println(out[0])   // classical
}

```

Output:

```

[pop indie R&B]
pop
classical

```

The `seen` map tracks which strings have already been added. The first time a value appears, it is added to `result` and recorded in `seen`; subsequent occurrences are skipped.

`result` is built with `append` onto a freshly allocated slice (`make([]string, 0, len(s))`), so it has its own backing array and is fully independent of `input`. Modifying an element of `result` does not affect `input`.

Pre-allocating with `cap=len(s)` is an optimization: in the worst case (all elements are distinct) `result` will grow to exactly `len(s)` elements, so pre-allocating avoids repeated reallocation.

## Chapter 8: Maps and Structs — Answers

**Exercise 1** (Think about it): In Java, you can use any object as a `HashMap` key as long as you override `hashCode()` and `equals()`. In Go, map key types must be **comparable** — they must support `==` at the language level. What are the advantages and disadvantages of Go's approach compared to Java's? Give one example of a Java key type that you cannot use directly as a Go map key, and explain how you would work around it.

### Go's approach — advantages:

- The constraint is enforced by the compiler at the type level with no runtime overhead. There is no risk of accidentally using a type as a key when its `==` semantics are broken or inconsistent.
- There is no equivalent of the Java footgun where you override `equals` but forget `hashCode`, or vice versa, causing `HashMap` to misbehave silently.
- The programmer does not need to implement any interface or write any boilerplate; any type that supports `==` just works.

### Go's approach — disadvantages:

- You cannot use slices, maps, or functions as keys, even if you have a meaningful notion of equality for them. In Java, if you implement `hashCode()` and `equals()` on a wrapper, you can use any object as a key.
- There is no way to customise the equality semantics for a key type. In Java, you can make two objects with different memory addresses compare as equal by overriding `equals()`. In Go, `==` on a struct compares field-by-field; you cannot override that.

**Example:** A `[]byte` slice is a common Java key (as a `byte[]` wrapped in a class with a custom `hashCode`). In Go, `[]byte` is not comparable and cannot be a map key directly. The idiomatic workaround is to convert the slice to `string` first: a `string` is comparable, and a `string([]byte{...})` conversion is a well-defined operation.

```
key := string([]byte{0x01, 0x02, 0x03})
m := map[string]int{}
m[key] = 42
```

This works because Go allows `string(b)` for any `[]byte b`, and the resulting string compares by byte content, which is the equality you typically want.

---

**Exercise 2** (What does this print?):

```
package main

import "fmt"

func main() {
    type Artist struct {
        Name string
    }
    type Song struct {
        Artist
        Title string
    }

    s := Song{
        Artist: Artist{Name: "Peso Pluma"},
        Title:  "La Beb  ",
    }

    s.Name = "Natanael Cano"
    fmt.Println(s.Title, "by", s.Name)
    fmt.Println(s.Artist.Name)
}
```

Output:

```
La Beb   by Natanael Cano
Natanael Cano
```

`s.Name = "Natanael Cano"` modifies the `Name` field of the embedded `Artist` value through the promoted field path. `s.Name` and `s.Artist.Name` refer to the same field; they are two ways to express the same memory location. After the assignment, both `s.Name` and `s.Artist.Name` return "Natanael Cano".

The first `fmt.Println` prints "La Beb   by Natanael Cano" because `s.Title` is "La Beb  " and `s.Name` (promoted from `Artist`) is now "Natanael Cano". The second `fmt.Println` prints "Natanael Cano" because `s.Artist.Name` is the same field accessed through the explicit path.

Key takeaway: embedding promotes fields; the promoted shorthand and the explicit path are identical at runtime.

---

**Exercise 3** (Calculation): Given the following map and operations, what does `len(m)` return after the final line?

```
m := map[string]int{
    "pop":      1,
    "rock":     2,
    "jazz":     3,
    "classical": 4,
}
delete(m, "rock")
delete(m, "country") // key does not exist
m["hip-hop"] = 5
m["jazz"] = 99
len(m) returns 4.
```

Trace:

1. After the literal, `m` has 4 entries: `pop`, `rock`, `jazz`, `classical`.
2. `delete(m, "rock")` removes `rock` — 3 entries remain.
3. `delete(m, "country")` is a no-op; `country` is not in the map — still 3 entries.
4. `m["hip-hop"] = 5` inserts a new key — 4 entries.
5. `m["jazz"] = 99` updates an existing key; this does **not** change the count — still 4 entries.

Final map: `{"pop": 1, "jazz": 99, "classical": 4, "hip-hop": 5}`. `len(m)` is 4.

---

**Exercise 4** (Where is the bug?): The following function is supposed to count how many tracks in a catalog have a BPM above a threshold, but it always returns 0.

```
package main

import "fmt"

func countFast(catalog map[string]int, threshold int) int {
    var count int
    for track := range catalog {
        if catalog[track] > threshold {
            count++
        }
    }
    return count
}

func main() {
    catalog := map[string]int{
        "Gasoline": 148,
        "Industry Baby": 160,
        "Numb": 72,
    }
    fmt.Println(countFast(catalog, 100))
}
```

The bug is that `countFast` is called with `nil` instead of `catalog`.

`main` creates `catalog` but then passes `nil` (not `catalog`) to `countFast`. A `nil` map has zero entries, so `for track := range nil` never executes, and `count` stays at 0.

Reading from a `nil` map is safe in Go — `catalog[track]` on a `nil` map returns 0 — but iterating over a `nil` map yields no iterations at all, so the loop body never runs.

The fix is to pass the actual map:

```
fmt.Println(countFast(catalog, 100))
```

With the correct call, the function returns 2 ("Gasoline" at 148 and "Industry Baby" at 160 both exceed the threshold of 100).

A secondary improvement: the loop uses `for track := range catalog` and then does a second map lookup `catalog[track]`. This works, but the idiomatic form uses the value directly from the range:

```
for _, bpm := range catalog {
    if bpm > threshold {
        count++
    }
}
```

This is one lookup per iteration instead of two.

---

**Exercise 5** (Write a program): Define a `Song` struct with fields `Title string`, `Artist string`, and `Plays int`. Build a slice of at least four `Song` values. Write `topN(songs []Song, n int) []Song` that returns the `n` most-played songs in descending order. Use `slices.SortFunc` and `cmp.Compare`.

```
package main
```

```
import (
    "cmp"
    "fmt"
    "slices"
)
```

```
type Song struct {
    Title  string
    Artist string
    Plays  int
}
```

```
// topN returns the n most-played songs in descending order by Plays.
// If n >= len(songs), all songs are returned.
func topN(songs []Song, n int) []Song {
    // work on a copy so the original slice is not reordered
    result := make([]Song, len(songs))
    copy(result, songs)

    // sort descending: negate the comparison to reverse order
    slices.SortFunc(result, func(a, b Song) int {
        return cmp.Compare(b.Plays, a.Plays) // b before a for descending
    })

    if n > len(result) {
```

```

        n = len(result)
    }
    return result[:n]
}

func main() {
    songs := []Song{
        {Title: "Blinding Lights",    Artist: "The Weeknd",    Plays: 4_000_000_000},
        {Title: "Stay",               Artist: "The Kid LAROI", Plays: 2_200_000_000},
        {Title: "Unholy",             Artist: "Sam Smith",    Plays: 2_000_000_000},
        {Title: "Save Your Tears",    Artist: "The Weeknd",    Plays: 1_800_000_000},
        {Title: "Golden Hour",        Artist: "JVKE",          Plays: 900_000_000},
    }

    fmt.Println("Top 3:")
    for _, s := range topN(songs, 3) {
        fmt.Printf("  %-28s %d plays\n", s.Title, s.Plays)
    }
}

```

Output:

```

Top 3:
Blinding Lights      4000000000 plays
Stay                 2200000000 plays
Unholy               2000000000 plays

```

Notes on the solution:

- `copy(result, songs)` prevents `slices.SortFunc` from mutating the caller's slice. If the caller passes a slice they intend to use in original order, an in-place sort would be a surprising side effect.
- `cmp.Compare(b.Plays, a.Plays)` with arguments reversed produces descending order. `cmp.Compare(a.Plays, b.Plays)` would sort ascending.
- The `n > len(result)` guard avoids a slice-bounds panic when the caller asks for more songs than exist.

---

## Chapter 9: Interfaces — Answers

**Exercise 1** (Think about it): Go's structural typing means any package can retroactively make its types satisfy an interface defined in any other package. In Java, if you want your `Song` class to satisfy a new interface `Playable` defined in a library you do not control, you must modify `Song`'s source. Explain how Go's approach changes the relationship between library authors and library users. What does this mean for extending types from packages you cannot modify?

Go's structural typing means that the author of a type and the author of an interface are completely decoupled. If library A defines `type Track struct { ... }` and later library B defines `interface Playable { Play() }`, and `Track` already has a `Play()` method, then `Track` satisfies `Playable` automatically — neither author needs to know about the other.

For types you cannot modify, the picture is similar: if the type already has the methods you need, you can use it directly where your interface is expected. If it does not, you have two options: wrap the type in your own struct that adds the missing methods (the adapter pattern), or define a new named type based on the original and add methods to that.

This is a fundamental philosophical difference. In Java, the relationship between a class and an interface is declared at write-time and embedded in the source. In Go, the relationship is discovered at compile-time by



the compiler — it emerges from what the type does, not from what it says it is. This makes Go code easier to extend and test, because you can define narrow interfaces in your own package that third-party types satisfy without any changes to the third-party source.

---

**Exercise 2** (What does this print?):

```
package main

import "fmt"

type Celsius float64
type Fahrenheit float64

func (c Celsius) String() string {
    return fmt.Sprintf("%.1f°C", float64(c))
}

func printTemp(v fmt.Stringer) {
    fmt.Println(v.String())
}

func main() {
    c := Celsius(37.5)
    f := Fahrenheit(99.5)
    printTemp(c)
    fmt.Println(f)
}
```

Output:

```
37.5°C
99.5
```

`Celsius` has a `String() string` method, so it satisfies `fmt.Stringer`. `printTemp` calls `v.String()` and passes the result to `fmt.Println`, which prints `37.5°C` followed by a newline.

`Fahrenheit` does **not** have a `String() string` method, so it does not satisfy `fmt.Stringer`. `fmt.Println(f)` formats `f` using the default verb for its underlying type, which is `float64`. The default formatting for a float is the shortest decimal representation that rounds back to the same value — here that is `99.5`. No degree symbol, no unit: just the number.

---

**Exercise 3** (Calculation): An interface value in Go stores two fields: a pointer to type information and a pointer to (or copy of) the data. Given a variable declared as `var r io.Reader = &bytes.Buffer{}`, how many distinct type/value components does `r` hold? If `r` is then assigned `nil`, describe the type and value components of the resulting interface value.

`r` holds **two** components: - **Type:** a pointer to the runtime type descriptor for `*bytes.Buffer`. - **Value:** a pointer to the `bytes.Buffer` value on the heap.

After `r = nil`, both components are set to `nil`: - **Type:** `nil` (no concrete type information). - **Value:** `nil` (no data pointer).

This is the **untyped nil** interface value. `r == nil` is `true` after this assignment.

Contrast this with the nil trap in the chapter: if instead you wrote `var buf *bytes.Buffer = nil; r = buf`, the type component would be `*bytes.Buffer` (non-nil) and the value component would be `nil`. That

interface value is **not** nil even though **buf** is nil.

---

**Exercise 4** (Where is the bug?):

```
package main

import "fmt"

type DBError struct{ code int }

func (e *DBError) Error() string { return fmt.Sprintf("db error %d", e.code) }

func connect(bad bool) error {
    var err *DBError
    if bad {
        err = &DBError{code: 500}
    }
    return err
}

func main() {
    e := connect(false)
    if e == nil {
        fmt.Println("connected OK")
    } else {
        fmt.Println("failed:", e)
    }
}
```

**The bug:** `connect` always returns a non-nil error, even when `bad` is false.

When `bad` is false, `err` is a nil `*DBError`. The `return err` statement wraps that typed nil in an error interface value. The interface value has type `*DBError` (non-nil) and value `nil`. Because the type component is non-nil, `e == nil` evaluates to false, and the program prints `failed: <nil>` instead of `connected OK`.

**The fix:**

```
func connect(bad bool) error {
    if bad {
        return &DBError{code: 500}
    }
    return nil // untyped nil: type=nil, value=nil --- this is a true nil error
}
```

Return `nil` directly rather than returning a typed nil pointer through an interface variable.

---

**Exercise 5** (Write a program):

```
package main

import (
    "fmt"
    "math"
)
```

```

type Shape interface {
    Area() float64 // returns the area of the shape
    Perimeter() float64 // returns the perimeter of the shape
}

type Rectangle struct {
    Width float64
    Height float64
}

func (r Rectangle) Area() float64 {
    return r.Width * r.Height
}

func (r Rectangle) Perimeter() float64 {
    return 2 * (r.Width + r.Height)
}

type Circle struct {
    Radius float64
}

func (c Circle) Area() float64 {
    return math.Pi * c.Radius * c.Radius
}

func (c Circle) Perimeter() float64 {
    return 2 * math.Pi * c.Radius
}

func printShapeInfo(s Shape) {
    fmt.Printf("Area: %.4f\n", s.Area())
    fmt.Printf("Perimeter: %.4f\n", s.Perimeter())
}

func main() {
    r := Rectangle{Width: 4.0, Height: 3.0}
    c := Circle{Radius: 5.0}

    fmt.Println("Rectangle:")
    printShapeInfo(r)

    fmt.Println("Circle:")
    printShapeInfo(c)
}

```

Output:

```

Rectangle:
Area:      12.0000
Perimeter: 14.0000
Circle:
Area:      78.5398
Perimeter: 31.4159

```

Both `Rectangle` and `Circle` satisfy `Shape` implicitly — no declaration required. `printShapeInfo` accepts any `Shape`, so adding a new shape (say, `Triangle`) requires only implementing `Area()` and `Perimeter()` on it; `printShapeInfo` does not change. This is the open/closed principle, Go style.